

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	X. Joshua
Name	192312188

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I X. Joshua – (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Question 1: End-to-End System Design

Design a scalable AI-based system using PySpark and RDD operations to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

1. Problem Understanding & Requirement Analysis

Social media platforms generate continuous, high-volume, unstructured data such as posts, likes, comments and shares. A recommendation system built on this data must handle high velocity, be fault-tolerant, and return personalised suggestions within seconds. The design objective is therefore a distributed pipeline that ingests live data, extracts features, applies intelligent agents for decision-making, and delivers recommendations without a single point of failure.

2. System Architecture

Layer	Components	Function
Data Ingestion Layer	Kafka / Flume connectors, Social media APIs	Continuously collects posts, comments and engagement events
Distributed Processing Layer	Spark Streaming, PySpark RDDs	Converts incoming micro-batches into RDDs and applies cleaning transformations
Storage Layer	HDFS / cloud object storage, NoSQL store	Persists raw and processed data, and user profile vectors
Intelligent Agent Layer	Sentiment Agent, Trend Agent, Recommendation Agent	Analyses processed RDDs and generates decisions
Presentation Layer	Dashboard / REST API	Delivers ranked recommendations to end users

3. Use of PySpark & RDD Operations

RDD / PySpark Operation	Purpose in the System
map()	Clean and normalise each post (remove noise, lower-case text)
flatMap()	Tokenise post text and split hashtags into individual terms
filter()	Discard spam, bot-generated or irrelevant posts
reduceByKey()	Aggregate hashtag / engagement counts per topic
join()	Combine user profile RDD with activity RDD
broadcast()	Distribute the trained recommendation model to all worker nodes
persist() / cache()	Reuse frequently accessed RDDs across multiple agent queries

4. Intelligent Agent Integration

Three cooperating agents operate on the processed RDDs. The Sentiment Agent classifies the emotional tone of each post; the Trend-Detection Agent flags topics whose engagement is rising sharply; the Recommendation

Agent matches a user's interest vector against trending, positively-received content using collaborative filtering. A lightweight Coordinator Agent merges the outputs of the three agents into a single ranked list before it reaches the presentation layer.

5. Scalability & Distributed Computing Design

Because Spark records the lineage of every RDD, a lost partition can be recomputed automatically without restarting the whole job, giving the pipeline fault tolerance. Horizontal scaling is achieved by adding worker nodes, and the streaming layer applies back-pressure control so that sudden spikes in social media activity (for example during a trending event) do not overwhelm the cluster. Periodic checkpointing keeps the RDD lineage graph short during long-running streaming jobs.

6. Data Flow & Processing Pipeline

Stage	Description
1. Ingest	Live posts and events are streamed into Spark via Kafka
2. Clean & Filter	RDD map/filter operations remove noise and irrelevant records
3. Feature Extraction	Text is tokenised and converted into feature vectors
4. Aggregation	reduceByKey aggregates engagement per topic/user
5. Agent Decision	Sentiment, Trend and Recommendation agents process the aggregated data
6. Delivery	Coordinator agent output is pushed to the dashboard/API

7. Innovation & Creativity

The system's originality lies in combining real-time sentiment shifts with trend detection so that recommendations adapt not just to what a user likes, but to how public opinion on a topic is currently changing, giving more timely and context-aware suggestions than a purely historical recommendation model.

Question 2: Distributed Intelligent Agent System

Develop a multi-agent AI system architecture that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

1. Problem Understanding & Requirement Analysis

Large-scale optimisation problems, such as citywide delivery-route planning or distributed resource scheduling, are too large for a single machine to solve directly and too interdependent to be split without coordination. The design goal is to divide the problem into partitions that can be optimised independently and in parallel, while still allowing agents to negotiate when their local decisions conflict with one another.

2. System Architecture

Layer	Components	Function
Data Layer	HDFS, operational databases	Stores orders, resources and constraints
Partitioning Layer	PySpark RDD partitioner	Splits the optimisation problem into independent sub-problems (e.g. by geographic zone)
Local Agent Layer	One Local Optimiser Agent per RDD partition	Runs a heuristic/greedy solver on its own partition in parallel
Negotiation Layer	Negotiation Agents (contract-net protocol)	Resolves conflicts where two partitions claim the same resource
Coordinator Layer	Global Coordinator Agent	Merges partial solutions and checks for convergence

3. Use of PySpark & RDD Operations

RDD / PySpark Operation	Purpose in the System
mapPartitions()	Runs each Local Agent's optimisation routine once per partition, avoiding per-record overhead
reduceByKey() / reduce()	Aggregates the local objective-function values into a global cost
broadcast()	Shares global constraints and the current best solution with every agent
accumulator	Tracks the running global cost across optimisation iterations
Iterative RDD transformations	Refines the solution over successive rounds until convergence

4. Intelligent Agent Integration

Each RDD partition is managed by its own Local Agent, which behaves independently and does not need to know about the entire problem. When two Local Agents' proposed assignments overlap (for example, the same delivery vehicle claimed by two zones), a Negotiation Agent applies a contract-net style bidding protocol so the resource goes to whichever agent gains more from it. The Coordinator Agent runs after every round, merging accepted proposals and deciding whether another optimisation round is needed.

5. Scalability & Distributed Computing Design

Because each partition is optimised independently, the workload is embarrassingly parallel and scales linearly with the number of worker nodes added to the cluster. If a worker fails mid-optimisation, Spark recomputes only the lost partition from its RDD lineage rather than restarting the whole job. Checkpointing is applied every few iterations to prevent the lineage graph from growing indefinitely during the iterative optimisation loop.

6. Data Flow & Processing Pipeline

Stage	Description
1. Decomposition	The optimisation problem is split into zone-based partitions
2. Local Optimisation	Each Local Agent solves its own partition in parallel
3. Conflict Detection	Overlapping resource claims are identified
4. Negotiation	Negotiation Agents resolve conflicts via bidding
5. Global Aggregation	Coordinator merges accepted proposals
6. Convergence Check	Repeat from step 2 or output the final solution

7. Innovation & Creativity

The design's novelty is in pairing a decentralised, negotiation-based multi-agent protocol with Spark's iterative distributed computation model, so that no single agent (or machine) is a bottleneck or a single point of failure while solving an NP-hard optimisation problem at city scale.

Question 3: Big Data AI Pipeline Design

Design a complete distributed AI pipeline using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

1. Problem Understanding & Requirement Analysis

Training machine learning models on big data requires a pipeline where preprocessing, feature engineering and model training can all run in parallel across a cluster, since the dataset is too large to fit or process on a single node. The pipeline must also tolerate node failures without losing progress on a long training job.

2. System Architecture

Stage	Components	Function
Ingestion	HDFS / cloud storage / Kafka	Loads raw batch or streaming data into the cluster
Preprocessing	PySpark RDD transformations	Cleans, deduplicates and handles missing values
Feature Engineering	RDD mapPartitions, encoders	Builds normalised, encoded feature vectors
Model Training	Spark MLlib on RDD/DataFrame	Trains the model in a distributed manner across partitions
Evaluation & Storage	Model registry, cloud storage	Validates and persists the trained model for deployment

3. Use of PySpark & RDD Operations

RDD / PySpark Operation	Purpose in the System
filter()	Removes null, duplicate or corrupt records
map()	Converts data types and normalises numeric fields
flatMap()	Expands multi-valued fields into individual records
distinct()	Removes duplicate entries before feature engineering
reduceByKey()	Computes aggregate statistics used for feature scaling
join()	Merges multiple data sources, e.g. user and transaction data
sample()	Creates the training/testing split
persist() / checkpoint()	Caches reused RDDs and truncates lineage during iterative training

4. Intelligent Agent Integration

While this pipeline is primarily a processing pipeline rather than an agent system, a lightweight Pipeline-Monitor Agent oversees each stage: it tracks data quality metrics after preprocessing, flags feature drift after feature engineering, and validates model accuracy after training, triggering a re-run of the relevant stage automatically if a threshold is not met.

5. Scalability & Distributed Computing Design

Data is partitioned across the cluster so that preprocessing and feature engineering run in parallel on each partition. Iterative ML algorithms such as gradient descent are checkpointed every few iterations to avoid excessively long RDD lineage chains, which would otherwise slow down fault recovery. If a node fails, Spark recomputes only the affected partitions from lineage rather than repeating the entire pipeline, and dynamic resource allocation adds executors when the cluster is under heavy load.

6. Data Flow & Processing Pipeline

Stage	Description
1. Ingestion	Raw data loaded from storage/stream
2. Cleaning	Nulls, duplicates and corrupt records removed
3. Feature Engineering	Encoding, normalisation and vector assembly
4. Train/Test Split	Data sampled into training and validation sets
5. Distributed Training	Model trained in parallel across partitions
6. Validation & Storage	Model evaluated and saved to the model registry

7. Innovation & Creativity

The pipeline reuses engineered features through a shared feature store so that multiple downstream models can draw on the same RDD-derived features without recomputing them, and checkpoint frequency is adjusted automatically based on lineage depth to balance fault-tolerance overhead against training speed.

Question 4: Real-Time Decision-Making System

Construct an AI-based system integrating RDD-based streaming data processing with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

1. Problem Understanding & Requirement Analysis

A smart-traffic management system must process continuous streams from traffic cameras, road sensors and GPS units and act on them within seconds — adjusting signal timings, suggesting reroutes and clearing paths for emergency vehicles. The design challenge is achieving low-latency distributed processing combined with automated, coordinated agent decisions.

2. System Architecture

Layer	Components	Function
Sensor/IoT Layer	Traffic cameras, road sensors, GPS	Continuously generates raw traffic events
Ingestion Layer	Kafka, Spark Structured Streaming	Buffers and streams events into the cluster
Processing Layer	PySpark micro-batch RDDs (DStreams)	Aggregates vehicle counts and speeds per junction
Intelligent Agent Layer	Signal Agent, Routing Agent, Emergency Agent	Makes real-time control decisions per junction
Actuation Layer	Signal controllers, navigation apps	Executes the agents' decisions in the physical world

3. Use of PySpark & RDD Operations

RDD / PySpark Operation	Purpose in the System
window()	Aggregates sensor readings over a sliding time interval
reduceByKey()	Computes vehicle count and average speed per junction
mapPartitions()	Calculates a congestion score per cluster partition
updateStateByKey()	Maintains rolling state such as average speed across batches

4. Intelligent Agent Integration

The Traffic Signal Agent adjusts green-light duration at each junction based on the current congestion score. The Routing Agent pushes alternate-route suggestions to navigation apps when a corridor is congested. The Emergency Agent monitors for ambulance/emergency vehicle signals and temporarily overrides normal signal timing to clear a path. Neighbouring Signal Agents also exchange short messages so that clearing congestion at one junction does not simply push it to the next junction (a coordinated "green wave").

5. Scalability & Distributed Computing Design

Distributed stream processing spreads sensor load across cluster nodes, allowing the system to scale from a single district to an entire city by adding more processing nodes. Streaming state is checkpointed regularly so that a node failure does not lose the rolling averages the agents depend on, and critical agent logic is replicated across nodes so there is no single point of failure for signal control decisions.

6. Data Flow & Processing Pipeline

Stage	Description
1. Sense	Sensor RDDs are ingested from cameras and GPS
2. Analyse	Congestion scores are computed per junction
3. Decide	Agents choose signal timing / reroute actions
4. Act	Decisions are sent to signal controllers and apps
5. Learn	Historical outcomes refine agent policy over time

7. Innovation & Creativity

Coordinating multiple junction agents rather than optimising each junction in isolation prevents congestion from simply shifting downstream, and the same streaming-plus-agent architecture generalises directly to healthcare, where the sensor layer becomes patient vital-sign monitors and the agents become triage and alert agents that flag deteriorating patients in real time.

Question 5: Cloud-Based Distributed AI Application

Design a cloud-enabled AI system using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

1. Problem Understanding & Requirement Analysis

A cloud-based analytics application must serve unpredictable and varying workloads cost-effectively: it should scale up automatically during heavy analytical load and scale down during idle periods, while its models keep improving as new data arrives, without needing to be retrained from scratch each time.

2. System Architecture

Layer	Components	Function
Cloud Infrastructure Layer	Managed Spark cluster, auto-scaling groups	Provides elastic compute for processing jobs
Data Layer	Cloud object storage / data lake	Stores raw and processed data, partitioned and versioned
Distributed Processing Layer	PySpark RDD/DataFrame jobs	Executes ETL and analytics workloads
Resource Management Layer	Cluster manager (YARN/Kubernetes)	Dynamically allocates or releases worker nodes
Intelligent Agent Layer	Monitoring, Analytics and Learning Agents	Tracks load, runs analytics, and retrains models incrementally

3. Use of PySpark & RDD Operations

RDD / PySpark Operation	Purpose in the System
mapPartitions()	Runs analytical computations in parallel batches
reduceByKey() / aggregateByKey()	Aggregates results across all cluster nodes
broadcast()	Distributes the current model/reference data efficiently
cache() / persist()	Speeds up repeated access during incremental retraining
coalesce() / repartition()	Adjusts partition count to match the current cluster size

4. Intelligent Agent Integration

The Monitoring Agent watches queue length, CPU and memory usage and triggers the cloud's autoscaling policy to add or remove worker nodes. The Analytics Agent executes the analytical queries and machine-learning inference requested by users. The Adaptive Learning Agent periodically retrains models incrementally on newly arrived RDD batches rather than performing a full retrain, so the system's predictions keep improving without downtime.

5. Scalability & Distributed Computing Design

Spark's dynamic resource allocation adds or removes executors in line with workload, while the Monitoring Agent drives the cloud provider's autoscaling to add or remove entire virtual machines. Data is replicated across availability zones for resilience, and processed results are checkpointed to cloud storage so that a cluster restart does not lose progress on long-running analytics jobs.

6. Data Flow & Processing Pipeline

Stage	Description
1. Ingestion	Data streamed or batch-loaded into the data lake
2. Dynamic Scaling	Cluster size adjusts to the incoming workload
3. Distributed Processing	PySpark RDD operations process the data
4. Agent Analytics	Analytics Agent produces insights/predictions
5. Adaptive Update	Learning Agent incrementally updates the model
6. Delivery	Insights are served via a dashboard/API

7. Innovation & Creativity

The system is cost-aware as well as accurate: the Monitoring Agent scales resources down during low demand to control cloud spend, while the Learning Agent keeps model quality rising continuously through incremental updates, avoiding the cost and downtime of periodic full retraining.